

# Mock Interview 2

Intermediate Round — Deep Technical + Scenario-Based

|            |                                                   |
|------------|---------------------------------------------------|
| Candidate  | Viraj Pawar                                       |
| Experience | 3.5 Years   Java Full Stack (Capgemini)           |
| Duration   | 50–60 Minutes                                     |
| Difficulty | ■■■ Medium-Hard                                   |
| Stack      | Java · Spring Boot · React · MySQL · Docker · K8s |

## Sections Covered in This Mock Interview

|           |                                                           |
|-----------|-----------------------------------------------------------|
| Section 1 | HR + Behavioral Round (12 Situational Questions)          |
| Section 2 | Core Java + Java 8/11/17 Advanced (13 Questions)          |
| Section 3 | Spring Boot Advanced (15 Questions + Scenario-Based)      |
| Section 4 | JPA + SQL Advanced (12 Questions + Complex Queries)       |
| Section 5 | Microservices Design + Patterns (12 Questions)            |
| Section 6 | DevOps — Kubernetes Deep Dive + Monitoring (10 Questions) |
| Section 7 | Coding Round (4 Medium Problems)                          |
| Section 8 | Rapid Fire + Cross Questions (12 Questions)               |
| Section 9 | Interviewer Evaluation Guide                              |

## ■ SECTION 1: HR + BEHAVIORAL — SITUATIONAL ROUND

**Q1.**

**Describe a time you had to learn a new technology quickly.**

◆ **Model Answer**

When our project decided to move to Kubernetes mid-sprint, I had no hands-on K8s experience. I dedicated my evenings for two weeks to Kubernetes in Action book and the official docs. I set up a local cluster using Minikube, deployed our services, hit problems, fixed them, and documented the lessons in a team wiki. Within two weeks, I wrote our first Helm chart and led the staging deployment. The team adopted it smoothly with zero rework.

**Q2.**

**Tell me about a time you disagreed with your manager and what happened.**

◆ **Model Answer**

My manager wanted to use Oracle DB for our Job Portal to align with another internal project. I respectfully raised concerns — our team's MySQL expertise was strong, Oracle licensing cost was significant, and the feature delta for our use case was negligible. I prepared a concise comparison document with TCO, migration risk, and team learning curve. After review, the manager agreed to stay with MySQL. The key was presenting data and business impact, not personal preference.

**Q3.**

**Tell me about a time a project failed and what you learned.**

◆ **Model Answer**

In my second year, we missed a deployment deadline because we underestimated the complexity of database schema migration for a live system. The rollback took 4 hours and impacted users. I learned: (1) Always test migrations on a production-like data volume. (2) Have a tested rollback script before any deployment. (3) Use tools like Flyway with undo scripts. I introduced these practices in subsequent projects and we never had a migration failure again.

**Q4.**

**How do you prioritize when you have multiple tasks with the same urgency?**

◆ **Model Answer**

I clarify urgency vs importance with stakeholders first — sometimes urgency is assumed, not real. Then I use impact analysis: which task unblocks the most people or has the highest business value? I communicate my plan to the Scrum Master and confirm. I also time-box tasks to avoid perfectionism blocking delivery.

**Q5.**

**Describe your experience with Agile and how you contributed to the team.**

◆ **Model Answer**

At Capgemini I worked in 2-week Sprints with daily standups, Sprint Planning, Review, and Retrospectives. I was proactive in Sprint Planning — I'd break down stories into technical tasks with estimates. In Retrospectives, I introduced 'Definition of Ready' criteria which reduced mid-sprint requirement changes by 40%. I also volunteered for demo preparation, which improved my client communication skills significantly.

**Q6.**

**Tell me about a time you mentored someone.**

◆ **Model Answer**

I informally mentored a junior developer joining our team. I set up weekly 30-minute knowledge sharing sessions covering Spring Boot patterns, Git workflows, and code review best practices. I gave constructive feedback on PRs and paired with him on complex features. Within 3 months, his code quality improved significantly and he was independently picking up medium-complexity stories. It also sharpened my own understanding because teaching forces clarity.

## ■ SECTION 2: CORE JAVA + JAVA 8/11/17 ADVANCED

Q1.

What are the new features in Java 17 that you use or know about?

◆ **Model Answer**

Sealed Classes: Restrict which classes can extend a superclass — great for domain modeling. Records: Compact, immutable data carriers — replaces boilerplate POJO/DTO classes. Pattern Matching for instanceof: Eliminates explicit casting. Text Blocks: Multi-line strings without escape sequences. Switch Expressions: Arrow syntax for compact switch. Virtual Threads (Java 21 preview / GA): Lightweight threads managed by JVM, not OS.

```
// Records (Java 16+): public record JobDTO(Long id, String title, Double salary)
{} // Pattern Matching instanceof (Java 16+): if (obj instanceof String s) {
System.out.println(s.length()); // no explicit cast! } // Sealed Classes (Java
17): public sealed class Shape permits Circle, Rectangle, Triangle {} public final
class Circle extends Shape { double radius; }
```

Q2.

What is the difference between `Collectors.groupingBy()` and `Collectors.partitioningBy()`?

```
// groupingBy: Groups into N categories by a classifier function Map<String,
List<Job>> byLocation = jobs.stream()
.collect(Collectors.groupingBy(Job::getLocation)); // partitioningBy: Groups into
exactly 2 groups: true/false Map<Boolean, List<Job>> remoteVsOnsite =
jobs.stream() .collect(Collectors.partitioningBy(Job::isRemote)); // Downstream
collector example: Map<String, Long> countByLocation = jobs.stream()
.collect(Collectors.groupingBy( Job::getLocation, Collectors.counting()));
```

Q3.

What is `CompletableFuture` and how did you use it?

◆ **Model Answer**

`CompletableFuture` is Java 8's non-blocking async computation framework. Unlike `Future`, you can chain callbacks and combine multiple async operations.

```
// Parallel fetch of user + job details: CompletableFuture<User> userFuture =
CompletableFuture.supplyAsync(() -> userService.findById(userId));
CompletableFuture<Job> jobFuture = CompletableFuture.supplyAsync(() ->
jobService.findById(jobId)); // Combine when both complete:
CompletableFuture<ApplicationDTO> result = userFuture.thenCombine(jobFuture,
(user, job) -> new ApplicationDTO(user, job)); ApplicationDTO dto = result.get();
// blocks until done // Chain transformations:
CompletableFuture.supplyAsync(this::fetchJobs) .thenApply(this::filterActive)
.thenAccept(System.out::println) .exceptionally(ex -> { log.error("Error", ex);
return null; });
```

Q4.

What is the difference between synchronized method, synchronized block, and `ReentrantLock`?

```
// Synchronized method: locks the entire method on "this" public synchronized void
increment() { count++; } // Synchronized block: finer-grained, lock on specific
object public void increment() { synchronized(lock) { count++; } // only this
section is locked } // ReentrantLock: more flexible, supports tryLock, timeout,
fairness private final ReentrantLock lock = new ReentrantLock(); public void
increment() { lock.lock(); try { count++; } finally { lock.unlock(); } // always
unlock! }
```

#### ◆ Model Answer

ReentrantLock advantages: tryLock() to avoid deadlock, lockInterruptibly(), timed lock attempts, fairness policy (longest-waiting thread gets lock first).

#### Q5.

#### What is the difference between HashMap, LinkedHashMap, TreeMap, and ConcurrentHashMap?

- HashMap: No ordering guarantee, O(1) average for get/put. Thread-unsafe.
- LinkedHashMap: Maintains insertion order (or access order). O(1) get/put. Good for LRU cache.
- TreeMap: Sorted by natural key order or Comparator. O(log n) operations. NavigableMap.
- ConcurrentHashMap: Thread-safe, segment-level locking (not entire map). Much better than synchronized HashMap.

#### Q6.

#### Explain Java Memory Model — Heap, Stack, MetaSpace.

#### ◆ Model Answer

Heap: Stores all object instances and class instances. Managed by GC. Divided into Young Gen (Eden + Survivor) and Old Gen. Stack: One per thread. Stores local variables, method call frames, primitive values. When a method is called, a new frame is pushed. StackOverflowError = too many recursive calls. MetaSpace (Java 8+, replaced PermGen): Stores class metadata, static variables, interned strings. Dynamically sized by default. OutOfMemoryError: Metaspace = too many dynamically loaded classes (e.g., excessive Reflection/proxies).

#### Q7.

#### What is a Deadlock? How do you prevent it?

```
// Deadlock example: // Thread1: locks A then tries to lock B // Thread2: locks B
then tries to lock A // -> circular wait = deadlock // Prevention: // 1. Always
acquire locks in the SAME ORDER across all threads // 2. Use tryLock() with
timeout: if (lock1.tryLock(1, TimeUnit.SECONDS)) { if (lock2.tryLock(1,
TimeUnit.SECONDS)) { // do work } else { lock1.unlock(); } } // 3. Use
higher-level concurrency constructs (Semaphore, Exchanger)
```

#### Q8.

#### What is the difference between fail-fast and fail-safe iterators?

#### ◆ Model Answer

Fail-fast: Throws ConcurrentModificationException if the collection is structurally modified during iteration. ArrayList, HashMap iterators are fail-fast. They track a modCount. Fail-safe: Works on a copy of the collection. No exception. CopyOnWriteArrayList, ConcurrentHashMap iterators are fail-safe. Downside: May not reflect latest changes.

#### Q9.

## Explain the Stream collect() method in depth.

```
// collect() is a terminal operation that reduces stream to a result // using a Collector: // Common collectors: List<String> list = stream.collect(Collectors.toList()); Set<String> set = stream.collect(Collectors.toSet()); String joined = stream.collect(Collectors.joining(", ", "[", "]")); Map<K,V> map = stream.collect(Collectors.toMap(k -> k, v -> v)); Map<Boolean, List<T>> partitioned = stream .collect(Collectors.partitioningBy(predicate)); Optional<T> max = stream.collect(Collectors.maxBy(comparator)); // Custom collector - immutable list: List<String> unmodifiable = stream .collect(Collectors.toUnmodifiableList());
```

### Q10.

#### What is var in Java 10+? Can you use it in all situations?

##### ◆ Model Answer

var is a local variable type inference keyword. The compiler infers the type from the right-hand side. Restrictions: Cannot be used for method parameters, return types, instance/class fields, or with null literal. Best used to reduce redundancy with long generic types: var map = new HashMap<>(); instead of HashMap<> map = ...

### Q11.

#### What are default and static methods in interfaces? Why were they added?

```
interface NotificationService { void send(String message); // Default method: has implementation, classes can override default void sendWithRetry(String message) { for (int i = 0; i < 3; i++) { try { send(message); return; } catch (Exception e) { /* retry */ } } } // Static method: utility method on the interface static NotificationService email() { return new EmailNotificationService(); } }
```

##### ◆ Model Answer

Added in Java 8 to allow adding new methods to interfaces without breaking all implementing classes (backward compatibility). This enabled the Streams API to be added to Collection without breaking existing code.

### Q12.

#### What is the difference between Iterator and ListIterator?

##### ◆ Model Answer

Iterator: Traverse in forward direction only. Available for all Collection types. Methods: hasNext(), next(), remove(). ListIterator: Bidirectional traversal. Only for List. Additional methods: hasPrevious(), previous(), add(), set(), nextIndex(). ListIterator allows modifying the list during traversal safely.

## ■ SECTION 3: SPRING BOOT ADVANCED + SCENARIOS

Q1.

**Scenario: Your API endpoint is slow — CPU is fine but response takes 3s. How do you investigate?**

◆ **Model Answer**

Step 1: Enable Spring Boot Actuator `/actuator/httptrace` to see recent HTTP request durations. Step 2: Enable SQL logging (`logging.level.org.hibernate.SQL=DEBUG`) to count queries per request. Step 3: Check for N+1 queries — often the most common cause. Step 4: Use Micrometer timer to instrument service method — `@Timed` annotation or manual `Timer.record()`. Step 5: Check for blocking I/O — external API calls, file reads. Step 6: Profile with Java Flight Recorder (JFR) for CPU hotspots. In my case, it was always either N+1 or a missing index.

Q2.

**How does Spring handle bean circular dependencies?**

◆ **Model Answer**

Spring Boot 2.6+ fails fast on circular dependencies by default. The fix: (1) Refactor to break the cycle — usually means one class is doing too much. (2) Use `@Lazy` on one of the injection points — delays bean creation. (3) Use setter injection instead of constructor injection for one of the beans. Constructor injection cannot handle circular deps because neither bean can be created first.

Q3.

**How does Spring Boot handle multiple implementations of the same interface?**

```
// Multiple implementations: @Service("emailNotifier") public class
EmailNotificationService implements NotificationService {} @Service("smsNotifier")
public class SMSNotificationService implements NotificationService {} // Inject a
specific one: @Autowired @Qualifier("emailNotifier") private NotificationService
notifier; // Inject all implementations: @Autowired private
List<NotificationService> allNotifiers; // Strategy pattern via
@ConditionalOnProperty: @Bean @ConditionalOnProperty(name="notify.channel",
havingValue="email") public NotificationService emailService() { return new
EmailNotificationService(); }
```

Q4.

**Explain Spring AOP. Give a real use case from your project.**

◆ **Model Answer**

AOP (Aspect Oriented Programming) allows cross-cutting concerns to be separated from business logic. Key concepts: Aspect (the concern), Advice (the action: Before, After, Around), Pointcut (where to apply), JoinPoint (the specific method execution). In my project, I used AOP for: (1) Request/Response logging — `@Around` advice on all controller methods logs method name, input params, execution time, and response status. (2) Metrics collection — `@Around` to record method timing sent to Prometheus.

```
@Aspect @Component public class ExecutionTimeAspect {
@Around("@annotation(Timed)") public Object logTime(ProceedingJoinPoint pjp)
throws Throwable { long start = System.currentTimeMillis(); Object result =
pjp.proceed(); long elapsed = System.currentTimeMillis() - start; log.info("{}
executed in {}ms", pjp.getSignature().getName(), elapsed); return result; } }
```

### Q5.

How do you write unit tests for a Spring Boot service using JUnit 5 and Mockito?

```
@ExtendWith(MockitoExtension.class) class JobServiceTest { @Mock private
JobRepository jobRepository; @InjectMocks private JobServiceImpl jobService; @Test
void findById_shouldReturnJob_whenExists() { // Arrange Job mockJob = new Job(1L,
"Java Dev", true);
when(jobRepository.findById(1L)).thenReturn(Optional.of(mockJob)); // Act JobDTO
result = jobService.findById(1L); // Assert assertNotNull(result);
assertEquals("Java Dev", result.getTitle()); verify(jobRepository,
times(1)).findById(1L); } @Test void findById_shouldThrow_whenNotFound() {
when(jobRepository.findById(99L)).thenReturn(Optional.empty());
assertThrows(ResourceNotFoundException.class, () -> jobService.findById(99L)); } }
```

### Q6.

What is @SpringBootTest vs @WebMvcTest vs @DataJpaTest?

#### ◆ Model Answer

@SpringBootTest: Loads full ApplicationContext. Slow but tests the real system. Use for integration tests. @WebMvcTest: Loads only web layer (Controllers, Filters). Mocks service layer. Fast — for testing controller logic in isolation. @DataJpaTest: Loads only JPA layer. Uses H2 in-memory by default. Fast — for testing repositories and custom queries.

■ Tip: Use @WebMvcTest for 80% of controller tests. It's fast and catches routing, validation, and serialization issues.

### Q7.

Scenario: You need to add rate limiting to your API. How would you implement it?

#### ◆ Model Answer

Options: (1) API Gateway level — Spring Cloud Gateway has built-in Redis Token Bucket rate limiting via RequestRateLimiter filter. Most efficient — rejects at gateway before hitting services. (2) Filter/Interceptor — Implement a HandlerInterceptor that checks a Bucket4J bucket per API key stored in Redis. (3) AOP — @Around advice on controller methods. My choice: API Gateway rate limiting with Redis is the most scalable — no changes to individual services needed.

### Q8.

How do you implement caching in Spring Boot?

```
// Enable caching: @EnableCaching // on @SpringBootApplication // Service layer:
@Service public class JobServiceImpl { @Cacheable(value = "jobs", key = "#id",
condition = "#id > 0") public JobDTO findById(Long id) { /* DB call */ }
@CacheEvict(value = "jobs", key = "#job.id") public JobDTO updateJob(Job job) { /*
updates DB */ } @CachePut(value = "jobs", key = "#result.id") public JobDTO
createJob(Job job) { /* saves to DB */ } } # application.properties:
spring.cache.type=redis spring.redis.host=localhost spring.redis.port=6379
```

### Q9.

How do you implement idempotency for POST endpoints?

#### ◆ Model Answer

The challenge: Retrying a POST (create job application) could create duplicates. Solution: Client sends a unique Idempotency-Key header (UUID) with each request. Server checks if this key exists in Redis. If yes, return the stored response. If no, process the request, store the response in Redis (TTL: 24h) with



the key, and return. This ensures retries are safe.

#### Q10.

**What is Feign Client and how did you use it for inter-service communication?**

```
@FeignClient(name = "user-service", // Eureka service name fallbackFactory =
UserClientFallbackFactory.class) public interface UserServiceClient {
@GetMapping("/api/users/{id}") UserDTO getUserById(@PathVariable Long id);
@PostMapping("/api/users/validate") boolean
validateToken(@RequestHeader("Authorization") String token); } // Usage in Job
Service: @Service public class ApplicationService { @Autowired UserServiceClient
userClient; public void apply(Long userId, Long jobId) { UserDTO user =
userClient.getUserById(userId); // ... } }
```

## ■ SECTION 4: JPA + SQL ADVANCED

Q1.

What is optimistic locking vs pessimistic locking in JPA?

```
// Optimistic Locking: No DB lock. Uses @Version column. // On concurrent update:
throws OptimisticLockException @Entity public class Job { @Version //
auto-incremented on each update private Long version; } // Pessimistic Locking:
Acquires DB-level lock @Query("SELECT j FROM Job j WHERE j.id = :id")
@Lock(LockModeType.PESSIMISTIC_WRITE) // SELECT FOR UPDATE Optional<Job>
findByIdForUpdate(@Param("id") Long id);
```

### ◆ Model Answer

Use optimistic for read-heavy scenarios where conflicts are rare. Use pessimistic for write-heavy, high-conflict scenarios (e.g., seat booking system).

Q2.

Write a SQL query to find the second highest salary among employers.

```
-- Method 1: Subquery with MAX SELECT MAX(salary) AS second_highest FROM users
WHERE salary < (SELECT MAX(salary) FROM users); -- Method 2: DENSE_RANK (handles
ties correctly) SELECT salary FROM ( SELECT salary, DENSE_RANK() OVER (ORDER BY
salary DESC) AS rnk FROM users ) ranked WHERE rnk = 2 LIMIT 1; -- Method 3: LIMIT
OFFSET (only if distinct salaries assumed) SELECT DISTINCT salary FROM users ORDER
BY salary DESC LIMIT 1 OFFSET 1;
```

■ Tip: Always use *DENSE\_RANK* in interviews — it handles ties and shows you know window functions.

Q3.

Explain SQL window functions with an example from your domain.

```
-- Rank employers within each city by number of jobs posted: SELECT c.name AS
company, c.city, COUNT(j.id) AS job_count, RANK() OVER ( PARTITION BY c.city ORDER
BY COUNT(j.id) DESC ) AS city_rank FROM companies c LEFT JOIN jobs j ON c.id =
j.company_id GROUP BY c.id, c.name, c.city; -- Running total of applications per
day: SELECT applied_date, COUNT(*) AS daily_count, SUM(COUNT(*)) OVER (ORDER BY
applied_date) AS running_total FROM applications GROUP BY applied_date;
```

Q4.

What is a CTE (Common Table Expression) and when would you use it?

```
-- Find companies with more than 10 active jobs and at least 100 applications:
WITH active_jobs AS ( SELECT company_id, COUNT(*) AS job_count FROM jobs WHERE
is_active = TRUE GROUP BY company_id HAVING COUNT(*) > 10
), application_summary AS ( SELECT j.company_id, COUNT(a.id) AS app_count FROM
jobs j JOIN applications a ON a.job_id = j.id GROUP BY j.company_id HAVING
COUNT(a.id) >= 100 ) SELECT c.name, aj.job_count, asumm.app_count FROM companies c
JOIN active_jobs aj ON c.id = aj.company_id JOIN application_summary asumm ON c.id
= asumm.company_id;
```

### ◆ Model Answer

CTEs improve readability and allow referencing the same subquery multiple times without repeating it. Use instead of nested subqueries for complex queries.

**Q5.**

**Explain EXPLAIN ANALYZE and how you'd use it to optimize a slow query.**

◆ **Model Answer**

EXPLAIN ANALYZE executes the query and shows the actual execution plan — time per step, row estimates, whether indexes were used. Key things to look for: (1) Seq Scan on a large table — missing index. (2) Using filesort or temporary — may need composite index or query rewrite. (3) Row estimate very off — stale table statistics, run ANALYZE TABLE. In my project I used EXPLAIN to discover that a LIKE '%java%' query was doing a full table scan. Replaced with FULLTEXT index and MATCH AGAINST for 10x speedup.

**Q6.**

**What is the difference between @GeneratedValue strategies in JPA?**

```
// IDENTITY: DB auto-increment. No pre-generation of IDs. @GeneratedValue(strategy = GenerationType.IDENTITY) // SEQUENCE: Uses DB sequence. Can pre-allocate blocks of IDs. // More efficient for batch inserts. @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "job_seq") @SequenceGenerator(name = "job_seq", sequenceName = "job_sequence", allocationSize = 50) // pre-allocate 50 IDs // TABLE: Portable but slow (uses a separate table for sequences) // UUID: Application-generated. Good for distributed systems. @GeneratedValue(generator = "UUID") @GenericGenerator(name = "UUID", strategy = "org.hibernate.id.UUIDGenerator")
```

## ■ SECTION 5: MICROSERVICES DESIGN PATTERNS

Q1.

**Explain the Strangler Fig pattern. When would you use it?**

◆ **Model Answer**

Strangler Fig is used to gradually migrate a monolith to microservices. Instead of a big-bang rewrite (high risk), you incrementally extract features into new services. An API Gateway or facade routes requests — new requests go to the new service, old ones still go to the monolith. Over time, the monolith 'dies' as all features are extracted. This minimizes risk and allows continuous delivery during migration.

Q2.

**What is the Outbox Pattern? Why is it needed?**

◆ **Model Answer**

The problem: After saving an entity to DB, you want to publish an event. If the DB commit succeeds but the event publish fails, you have inconsistency. The Outbox Pattern: Instead of publishing directly, write the event to an 'outbox' table in the SAME DB transaction. A separate poller reads the outbox and publishes to the event broker. Once published, mark the outbox record as processed. This guarantees at-least-once event delivery.

Q3.

**What is service mesh? Have you worked with Istio or Linkerd?**

◆ **Model Answer**

A service mesh is an infrastructure layer that handles service-to-service communication — mTLS (mutual TLS), load balancing, retries, circuit breaking, observability. Implemented as sidecar proxies (Envoy in Istio). I haven't used Istio in production, but I understand the architecture: a control plane (Istiod) configures sidecar proxies in each pod. Benefits over application-level patterns: Language-agnostic, no code changes needed for retries/timeouts/tracing.

■ *Tip: It's perfectly fine to say you know the concept but haven't used it in production — BUT show you understand WHY it exists.*

Q4.

**How do you handle versioning of REST APIs?**

◆ **Model Answer**

Three approaches: (1) URI Versioning: `/api/v1/jobs` — most visible and cache-friendly. My preferred approach. (2) Header Versioning: Accept: `application/vnd.jobportal.v2+json` — cleaner URIs but less discoverable. (3) Query Parameter: `/api/jobs?version=2` — simple but pollutes query string. API versioning strategy: Support at least 2 versions simultaneously with a 6-month deprecation notice. Document breaking changes clearly in changelog.

Q5.

**How do you ensure data consistency in a distributed system without 2PC?**

◆ **Model Answer**

Use the BASE model (Basically Available, Soft state, Eventually consistent) instead of ACID. Techniques: (1) SAGA Pattern — compensating transactions for rollback. (2) Idempotent event handlers — processing the same event twice has no side effects. (3) Event sourcing — state derived from

immutable event log, can replay events to rebuild state. (4) Outbox pattern for reliable event publishing. Accept that consistency is eventual — design the UI/UX to handle this (e.g., 'Application submitted — you'll receive confirmation shortly').

## ■ SECTION 6: DEVOPS — KUBERNETES DEEP DIVE

**Q1.**

**What is a Kubernetes Ingress? How is it different from a Service?**

◆ **Model Answer**

Service: Exposes a set of Pods within the cluster (ClusterIP) or externally (NodePort/LoadBalancer). One service per application. Ingress: An API object that manages external HTTP/HTTPS routing. One Ingress can route to multiple services based on host/path rules. Requires an Ingress Controller (nginx-ingress, Traefik). Ingress also handles TLS termination. Example: /api/jobs → job-service, /api/users → user-service — all through one Ingress.

**Q2.**

**What are Kubernetes liveness and readiness probes?**

```
# Kubernetes deployment spec: livenessProbe: httpGet: path:
/actuator/health/liveness port: 8080 initialDelaySeconds: 30 # wait for JVM
startup periodSeconds: 10 failureThreshold: 3 # restart after 3 failures
readinessProbe: httpGet: path: /actuator/health/readiness port: 8080
initialDelaySeconds: 20 periodSeconds: 5 failureThreshold: 3 # remove from LB
until healthy
```

◆ **Model Answer**

Liveness: Is the container alive? If not, K8s restarts it. Readiness: Is the container ready to serve traffic? If not, K8s removes it from the Service endpoints (no traffic sent). Spring Boot Actuator provides separate /actuator/health/liveness and /actuator/health/readiness endpoints.

**Q3.**

**How did you monitor your microservices? Describe your observability stack.**

◆ **Model Answer**

The three pillars of observability: (1) Metrics: Micrometer + Prometheus collects JVM, HTTP, custom metrics. Grafana dashboards visualize them. Alerts (PagerDuty) on error rate > 5% or p99 latency > 2s. (2) Logs: Structured JSON logging with Logback. Fluentd ships to Elasticsearch. Kibana for search and visualization. (3) Traces: Spring Cloud Sleuth adds trace/span IDs to every log line. Zipkin collects and visualizes distributed traces across services.

**Q4.**

**What is a Kubernetes StatefulSet? When would you use it?**

◆ **Model Answer**

StatefulSet manages Pods that need persistent, stable identity — stable network names (pod-0, pod-1), stable persistent storage (PVC per pod), ordered deployment/scaling. Use cases: Databases (MySQL, Cassandra), Kafka brokers, Redis clusters. Deployments are for stateless apps — Pods are interchangeable. StatefulSets are for stateful apps where order and identity matter. In my project, MySQL was deployed as a StatefulSet with a PersistentVolumeClaim per replica.

## ■ SECTION 7: CODING ROUND — MEDIUM PROBLEMS

### Problem 1 — Valid Parentheses (Easy-Medium)

Given a string containing '(', ')', '{', '}', '[', ']', determine if it's valid. Open brackets must be closed in correct order.

```
public boolean isValid(String s) { Deque<Character> stack = new ArrayDeque<>();
Map<Character, Character> map = Map.of( ')', '(', '}', '{', ']', '[' ); for (char
c : s.toCharArray()) { if (!map.containsKey(c)) { stack.push(c); // opening
bracket } else { if (stack.isEmpty() || stack.pop() != map.get(c)) return false; }
} return stack.isEmpty(); }
```

*O(n) time, O(n) space. Follow-up: What if you have custom bracket types?*

### Problem 2 — Longest Substring Without Repeating Characters (Medium)

Given 'abcabcbb', return 3 (for 'abc').

```
public int lengthOfLongestSubstring(String s) { Map<Character, Integer> charIndex
= new HashMap<>(); int maxLen = 0; int left = 0; for (int right = 0; right <
s.length(); right++) { char c = s.charAt(right); if (charIndex.containsKey(c) &&
charIndex.get(c) >= left) { left = charIndex.get(c) + 1; // shrink window }
charIndex.put(c, right); maxLen = Math.max(maxLen, right - left + 1); } return
maxLen; }
```

*Sliding window pattern. O(n) time. Follow-up: For k distinct characters?*

### Problem 3 — Flatten Nested List (Medium)

Given [[1,1],2,[1,1]], return [1,1,2,1,1]. Flatten any level of nesting.

```
public List<Integer> flatten(List<Object> list) { List<Integer> result = new
ArrayList<>(); flattenHelper(list, result); return result; } private void
flattenHelper(List<Object> list, List<Integer> result) { for (Object item : list)
{ if (item instanceof Integer) { result.add((Integer) item); } else if (item
instanceof List) { flattenHelper((List<Object>) item, result); // recurse } } } //
Iterative using Stack: public List<Integer> flattenIterative(List<Object> input) {
List<Integer> result = new ArrayList<>(); Deque<Object> stack = new
ArrayDeque<>(input); while (!stack.isEmpty()) { Object item = stack.pop(); if
(item instanceof Integer) result.add(0, (Integer) item); else
stack.addAll((List<Object>) item); } return result; }
```

### Problem 4 — LRU Cache (Medium-Hard)

Design a data structure that follows LRU (Least Recently Used) eviction. Implement get(key) and put(key, value) in O(1).

```
class LRUCache { private final int capacity; // LinkedHashMap: insertion-order +
access-order = perfect for LRU private final Map<Integer, Integer> cache; public
LRUCache(int capacity) { this.capacity = capacity; // true = access-order (most
recently used moves to tail) this.cache = new LinkedHashMap<>(capacity, 0.75f,
true) { @Override protected boolean removeEldestEntry(Map.Entry<Integer, Integer>
e) { return size() > capacity; // evict LRU } }; } public int get(int key) { return
cache.getOrDefault(key, -1); } public void put(int key, int value) {
cache.put(key, value); } }
```

*Follow-up: Implement without LinkedHashMap using HashMap + custom Doubly Linked List.*



## ■ SECTION 8: RAPID FIRE + CROSS QUESTIONS

|            |                                                                        |                                                                                                                                                                    |
|------------|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Q1</b>  | <b>What is a memory leak in Java?</b>                                  | Objects referenced in long-lived collections (static maps, caches) but never removed. GC can't collect them. Tool: JVisualVM, JProfiler to detect.                 |
| <b>Q2</b>  | <b>Difference: @Mock vs @Spy in Mockito?</b>                           | @Mock: Creates a complete mock — all methods return default.<br>@Spy: Wraps real object — real methods called unless explicitly stubbed.                           |
| <b>Q3</b>  | <b>What is the Open/Closed Principle?</b>                              | Classes should be open for extension, closed for modification. Add new behavior via new classes/interfaces, not by modifying existing ones.                        |
| <b>Q4</b>  | <b>What is a race condition?</b>                                       | When outcome depends on thread scheduling order — two threads modify shared state without synchronization.                                                         |
| <b>Q5</b>  | <b>What is eventual consistency?</b>                                   | System guarantees that if no new updates are made, all replicas will eventually converge to the same value. Used in distributed systems.                           |
| <b>Q6</b>  | <b>What is Spring's @Lazy annotation?</b>                              | Delays bean initialization until first usage. Useful to break circular dependencies or reduce startup time for rarely used beans.                                  |
| <b>Q7</b>  | <b>HTTP 401 vs 403?</b>                                                | 401 Unauthorized: Not authenticated (no/invalid token). 403 Forbidden: Authenticated but lacks permission.                                                         |
| <b>Q8</b>  | <b>What is a Docker volume?</b>                                        | Persistent storage for containers. Data in volumes survives container restart/deletion. Mounted from host or managed by Docker.                                    |
| <b>Q9</b>  | <b>What is @EnableJpaRepositories?</b>                                 | Enables Spring Data JPA repositories scanning in the specified package. Auto-enabled by Spring Boot.                                                               |
| <b>Q10</b> | <b>What is a Kubernetes taint and toleration?</b>                      | Taint: Prevents pods from being scheduled on a node.<br>Toleration: Allows a pod to be scheduled on a tainted node. Used to dedicate nodes for specific workloads. |
| <b>Q11</b> | <b>What is spring.jpa.open-in-view?</b>                                | If true (default), keeps EntityManager open for the full request (including view rendering). Can cause N+1 issues. Best practice: set to false.                    |
| <b>Q12</b> | <b>Difference: @SpringBootApplication vs @EnableAutoConfiguration?</b> | @SpringBootApplication = @EnableAutoConfiguration + @ComponentScan + @Configuration. It's a convenience meta-annotation.                                           |

## ■ SECTION 9: INTERVIEWER EVALUATION GUIDE

### Technical Depth Signals

- Mentions internals (HashMap TreeNode, HikariCP, Hibernate dirty checking) without being asked.
- Uses correct terminology: EntityManager, ProceedingJoinPoint, TokenBucket, AllocationSize.
- Describes failures and what they learned — shows production experience.
- Knows not just WHAT but WHEN NOT TO USE something (e.g., Eager loading is fine for ManyToOne).
- Mentions observability proactively — logging, metrics, traces — without being prompted.

### Differentiation Factors for Mid-Level (3-5 yrs)

- Can design a solution end-to-end: API contract → service → repository → DB schema.
- Mentions trade-offs spontaneously: 'I chose X but Y would work if the requirement changes to...'
- Knows when NOT to use microservices — can articulate the distributed systems tax.
- Has debugged real production issues — not just knows theory, has fixed things under pressure.
- Can write clean, edge-case-aware code without being prompted for edge cases.

### Accenture-Specific Tips

- Accenture values client-readiness: dress well, speak clearly, and show confidence.
- Emphasize your ability to work in diverse, multi-team environments.
- Show enthusiasm for learning new domains — Accenture rotates across industries.
- Mention any certifications or ongoing learning: AWS, Spring certifications, Udemy courses.
- Ask about the project/team at the end — shows genuine interest, not just job-hunting.